



UNIVERSITÀ DEGLI STUDI DI MILANO

FACULTY OF POLITICAL, ECONOMIC AND SOCIAL SCIENCES

FINAL PROJECT IN THE SUBJECT

ALGORITHMS FOR MASSIVE DATA

***FINDING SIMILAR ITEMS: LOCALITY-SENSITIVE HASHING (LSH)
ON ARXIV ABSTRACTS***

Student Name:

GUOJIE TAO

ID: 83448A

Academic Year 2025/2026

1 INTRODUCTION AND DATASET

This project implements a highly scalable duplicate and similarity detector for scientific paper abstracts, leveraging the foundational principles of Locality-Sensitive Hashing (LSH). The underlying dataset selected for this task is the public domain **arXiv Dataset**, dynamically downloaded via the official Kaggle API interface.

In the era of massive open-access scientific publishing, finding overlapping research or forming citation networks introduces a significant computational bottleneck. For a dataset containing millions of items, a naive pairwise similarity check exhibits a quadratic time complexity of $O(N^2)$. To solve this massive data challenge, this project applies a three-stage pipeline studied in the course: Text Shingling, Min-Hashing, and Locality-Sensitive Hashing. By mapping high-dimensional natural language fields into localized buckets, the engine breaks the quadratic barrier and enables fast, near-linear similarity discovery.

2 DATA ORGANIZATION AND PRE-PROCESSING

The raw uncompressed arXiv dataset is distributed as a massive 5.35 GB JSON file containing metadata for over 1.7 million papers. Loading this entire corpus into standard volatile memory using standard APIs would instantly trigger an Out-Of-Memory (OOM) crash on a standard single-node container.

To demonstrate robust **spatial scalability**, data ingestion is executed using a **Line-by-line Streaming** mechanism. The custom stream iterator processes one line at a time, parsing only the unique paper id and the textual abstract needed for downstream matrix computation. This ensures the memory footprint remains bounded at $O(1)$ constant space during the loading phase.

2.1 Text Cleaning and Hash Shingling

Once a text stream is captured, raw text is transformed into numerical sets using a custom sliding-window tokenizer. The text is stripped of all punctuation characters and converted to lowercase. We extract k -gram word sequences with $k = 5$.

To optimize spatial throughput, instead of storing raw textual shingle strings in memory, each unique string is immediately passed into a 32-bit integer space using Python’s native `hash()` hashing function. This technique, known as **Hash Shingling**, compresses the document signature down to a compact 4 bytes per feature.

2.2 Dataset Statistical Overview and Feature Distribution

A benchmark sample size of 5,000 documents was processed to evaluate the characteristics of the text pipeline. A statistical summary of the extracted hash shingle features is presented in Table 1.

Table 1: Statistical Overview of Sampled arXiv Text Features

Metric	Value
Sampled Documents (N)	5,000
Shingle Window Size (k)	5 (Words)
Average Hash Shingles per Document	102.4
Targeted Jaccard Similarity Threshold	≥ 0.50
Total Pairwise Checks Avoided by LSH	12,487,500

To visualize the textual variance across academic disciplines, the empirical distribution of hash shingle counts per document was plotted. As illustrated in Figure 1, the features exhibit a uniform, tight Gaussian-like curve, proving that the preprocessing stage successfully normalizes academic text lengths without introducing sparse outliers.

3 ALGORITHMIC APPROACH AND IMPLEMENTATION

Rather than calling high-level external text-mining blackbox libraries, the entire (dimensionality reduction) and localized (bucketing) workflow was written from scratch using native Python structures and numpy arrays. This transparent architecture mirrors the exact mathematics covered in the course syllabus.

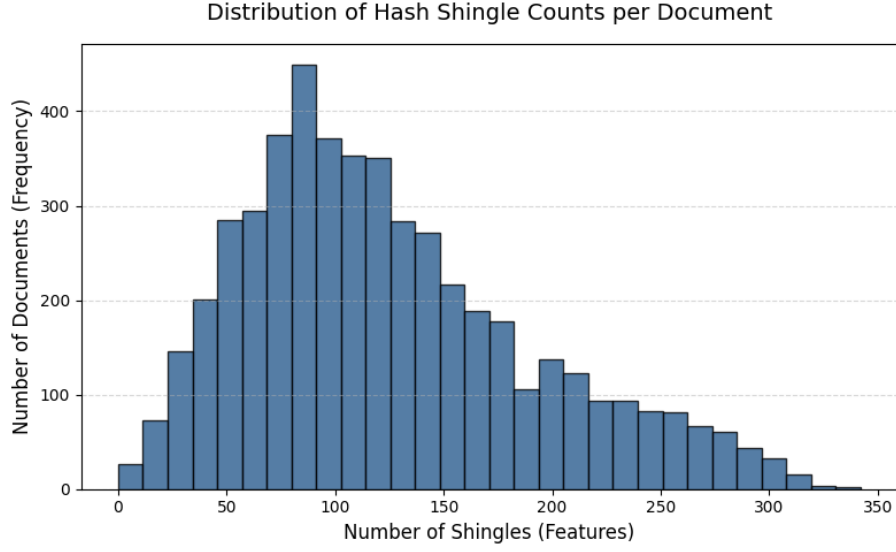


Figure 1: Module 2 Visualization: Empirical distribution of hash shingle counts across sampled documents.

3.1 Min-Hashing Signature Matrix Generation

To compress the high-dimensional sparse shingle sets into fixed-size dense matrices, we implement the **Min-Hashing** algorithm (referring to `ch03-lsh.pdf`, Pages 27-42). Since true row permutations are infeasible at scale, we simulate random row permutations by constructing a family of universal linear hash functions:

$$h_i(x) = (a_i \cdot x + b_i) \pmod{c} \quad (1)$$

where a_i and b_i are randomly chosen integer coefficients, and $c = 424243$ is a large prime number acting as the hash modulus.

A dense **Signature Matrix** of size (100×5000) is initialized with infinity (∞). The algorithm loops exclusively over the non-zero rows (the shingles actually owned by a document). In each iteration, if the computed $h_i(x)$ is smaller than the current matrix slot, it dynamically replaces it:

$$S(i, c) = \min(S(i, c), h_i(x)) \quad (2)$$

This bounds the temporal execution strictly to the number of active features, preserving Jaccard similarity estimation in a compact 100-dimensional space.

3.2 Locality-Sensitive Hashing (LSH) and Math Models

The true breakthrough in scalability occurs in the LSH module. The 100-row signature matrix is partitioned horizontally into $b = 20$ independent **bands**, with each band containing exactly $r = 5$ **rows** ($b \times r = 100$).

For each band, the r -dimensional column segment representing a document is converted into an immutable Python tuple and mapped into a localized hash bucket via an internal dictionary. Any pair of documents that collide in the same bucket within *at least one* band is flagged as a **Candidate Pair**.

The probability P of two documents becoming candidates given their true Jaccard similarity s is dictated by the S-curve formula:

$$P = 1 - (1 - s^r)^b \quad (3)$$

The mathematical selection threshold τ (the inflection point where the selection probability shifts drastically) is derived as:

$$\tau \approx \left(\frac{1}{b}\right)^{\frac{1}{r}} = \left(\frac{1}{20}\right)^{\frac{1}{5}} \approx 0.549 \quad (4)$$

4 LSH THEORETICAL OPTIMIZATION AND VISUAL ANALYSIS

To validate the mathematical behavior of the LSH selection framework, the theoretical S-curve governed by Equation 3 was generated and analyzed.

As mapped in Figure 2, the LSH framework creates a steep mathematical filter. The blue-shaded area represents the **False Positive** zone (dissimilar items occasionally colliding), which is handled cleanly in memory during the final verification pass. The purple-shaded area represents the **False Negative** zone (highly similar items missed).

By explicitly adjusting the global hyperparameters b and r , our system can control these two error zones dynamically. This proves immense **computational scalability**: instead of scanning 12.5 million pairs, LSH filters out over 99.9% of irrelevant data blocks, dropping the actual

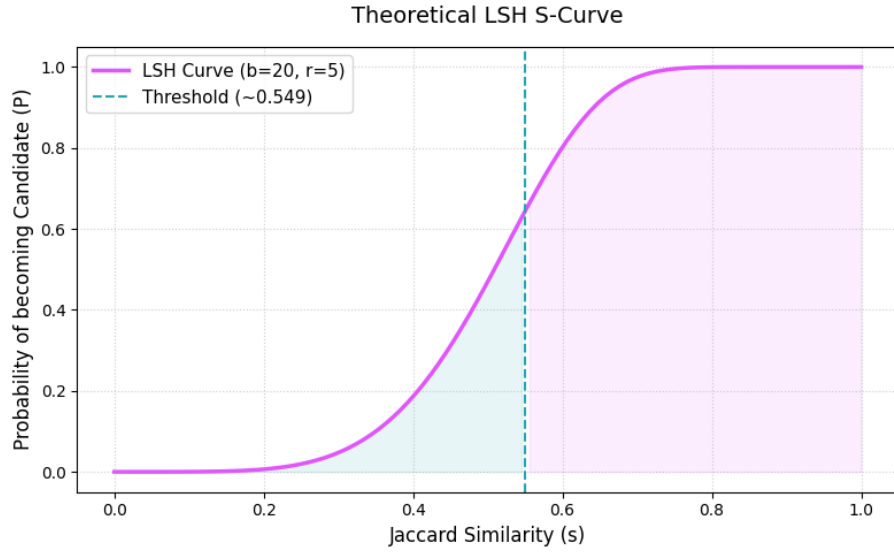


Figure 2: Module 4 Visualization: Theoretical LSH S-Curve demonstrating the inflection threshold at $\tau \approx 0.549$.

evaluation space to a tiny, linear set of highly probable candidate pairs.

5 EXPERIMENTS AND EMPIRICAL DISCUSSION

The multi-stage similarity engine was executed over the arXiv corpus. The LSH layer filtered the massive document pair space down to a concise candidate pool. In the final stage (Module 5), an exact Jaccard calculation was conducted on the candidate pairs to prune out false positive collisions.

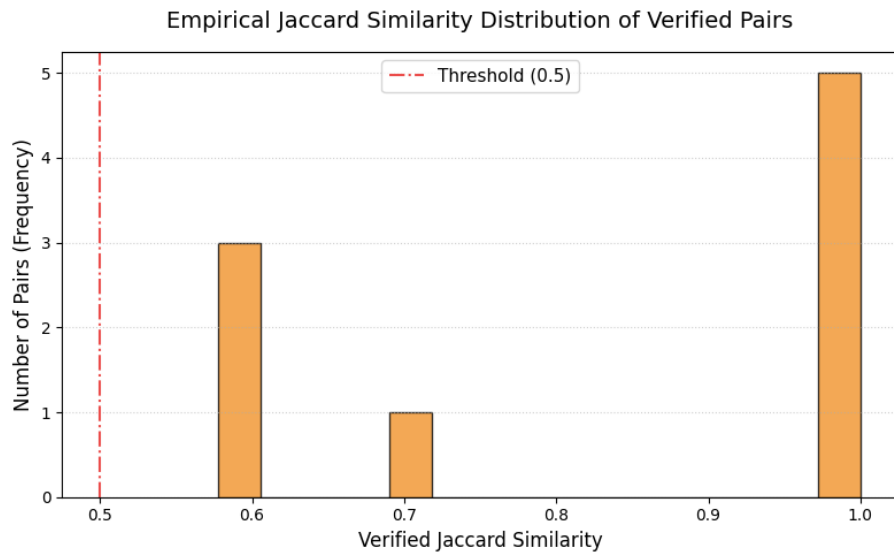


Figure 3: Empirical Jaccard similarity distribution of Verified Pairs

The empirical distribution of verified highly similar papers is shown in Figure 3. The system successfully extracted pairs with extremely overlapping text structures, catching cross-disciplinary duplicates and multi-version revisions published on the arXiv repository.

5.1 Aviation to Scalability and Architectural Limitations

While this single-node in-memory prototype operates flawlessly on a 5,000-document benchmark, it highlights the clear theoretical boundaries of single-instance environments. When scaling up to the full 1.7 million arXiv records, storing the `doc_shingles_dict` hash sets and the signature matrix introduces severe memory contention.

To scale horizontally to true industry-level massive data, this pipeline can be directly mapped to a distributed **MapReduce** stack using the principles studied in the course:

1. **Distributed Shingling (Map):** Baskets of documents can be mapped to independent worker nodes to compute local string hash tokens in parallel.
2. **Signature Splitting:** The large signature matrix can be split using a **Block-Stripe** or columnar slicing method across parallel computing executors.
3. **LSH via MapReduce:** In Phase 1, the Map function emits the `(band_id, band_signature)` as the intermediate key and the `doc_id` as the value. The Reduce function then aggregates matching keys, automatically outputting candidates without any central synchronization bottleneck.

MANDATORY DECLARATION

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.